ARIA

Autonomous Reinforcement Irrigation Agent

Deep Reinforcement Learning for Precise Tomato Crop Irrigation

---

Tiya (135) & Vaidhav (136) | Reinforcement Learning | April 30, 2026

Problem Statement

What problem are we solving?

- Traditional irrigation uses **static and timerbased schedules**. This means crops are watered on a fixed clock regardless of soil moisture, rainfall or temperature
- The system is completely **"blind"**, which means it waters mid monsoon, over pumps until the motor burns out, and starves crops during droughts

Why does it matter?

- Agriculture consumes ∼70% of global freshwater waste (Over Exploitation)
- Climate variability (extreme droughts and monsoons) is increasing year on year
- There now exist low cost IoT sensors and compute, which make real time AI feasible

Our solution: An RL agent trained on a custom simulation (FarmEnv) that reads live soil and weather telemetry every day and makes optimal irrigation decisions maximising crop yield within finite water and hardware constraints.

RL Formulation: Environment (FarmEnv)

Three Phase: Seasonal Simulation

- **Spring (0–30):** 25°C, 50% humidity
- **Drought (30–60):** 35°C, 20% humidity
- **Monsoon (60–90):** 22°C, 80% humidity

Each phase randomised via Gaussian noise.

Evaporation: Daily soil moisture loss:

$$\text{evap} = \underbrace{0.05}_{\text{base}} + \underbrace{0.08\, T_n}_{\text{heat}} + \underbrace{0.04\,(1 - H_n)}_{\text{dry air}}$$

Plant Growth: Daily increment:

$$\Delta g = r \cdot D(t) \cdot R(\rho) \cdot S(H, T, V)$$

- $r = 0.016$ — This is the fixed max daily rate
- $D(t) = \exp\left(-\frac{(t-0.5)^2}{0.157}\right)$ — Gaussian bell peaking at Day 45 (peak growth phase)
- $R(\rho)$ — storm gate where if rain > 0.8, we see canopy damage
- $S = (0.5\sqrt{H} + 0.3\sqrt{T} + 0.2\sqrt{V})^2$ — Weighted Generalised Mean stress; soil gets 50% weight as the only agent-controlled factor

RL Formulation — State, Action & Reward

State Space $\mathcal{S}$ — 8D, normalised $[0, 1]$

1. Soil Moisture — opt: 0.55
2. Water Tank Level
3. Growth Score
4. Normalised Day — day/90
5. Temperature
6. Humidity
7. Rain Now
8. Rain Forecast — 24h

Action Space $\mathcal{A}$

- **0** — Do nothing
- **1** — Irrigate: $+0.20$ soil moisture, -5% tank

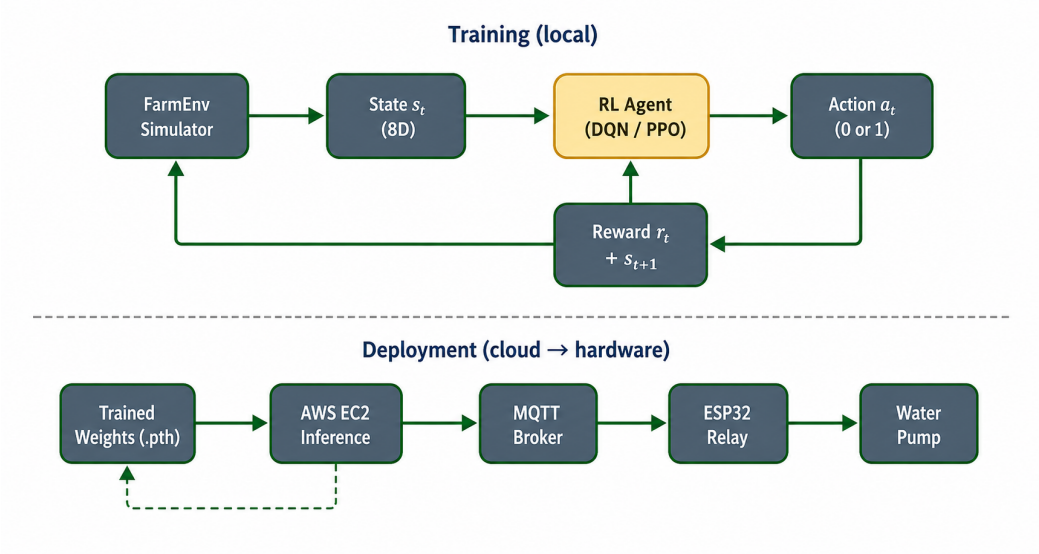
Reward Function $\mathcal{R}$

Signal	Value	Condition
Growth below curve	-5.0	actual $<$ expected
Growth above curve	$+\delta$	actual $>$ expected
Pump activation	-0.05	Every irrigation
Empty tank pump	-5.0	Tank $< 5\%$
Dry soil, no water	-2.0	Soil $< 30\%$, action $= 0$
Harvest	$+\mathbf{100}$	Day 89, growth $\geq 80\%$

Methodology — Algorithms Evaluated

Algorithm	Category	What it does	Outcome
Q-Learning	Tabular	Discrete Q-table lookup	Tabular ceiling
SARSA	Tabular	On-policy Q-table update	Tabular ceiling
REINFORCE-vanilla	Policy Gradient	Monte Carlo gradient, no baseline	High variance, poor convergence
REINFORCE-baseline	Policy Gradient	Policy gradient with value baseline	Starvation — never irrigates
Actor-Critic	Policy Gradient	Shared value and policy network	Pump-spammer — always irrigates
DQN	Deep RL	MLP approximates $Q^*(s, a)$ with replay	**Optimal policy**
PPO	Deep RL	Clipped surrogate actor-critic	Near-optimal

Methodology — System Workflow



Implementation

Local Training

- DQN & PPO train against `farm_env.py`, 500 episodes (45,000 days)
- We use simulated soil mositure value and etc

Policy Export

- `best_dqn.pth` transferred to AWS EC2

Cloud Inference

- `cloud_inference.py` subscribes to ESP32 via MQTT
- Queries OpenWeather API, builds live 8D state vector
- Publishes 0/1 relay command to Arduino pump

Key Libraries

- **PyTorch** — DQN & PPO neural networks
- **NumPy** — 8D state normalisation & clipping
- **Paho-MQTT** — ESP32 $\leftrightarrow$ EC2 communication

Supporting Stack

- **SQLite3** — edge telemetry database
- **Ultralytics / OpenCV** — YOLO tomato detection
- **Matplotlib** — 7-Agent Showdown graphs

Results I — Reward Curves

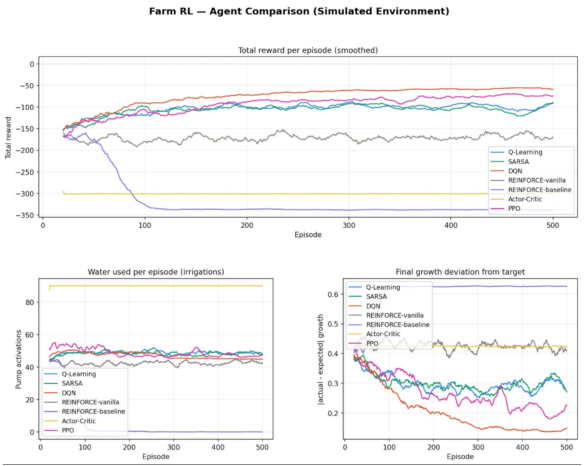


Figure: Final Run Reward Curvex

Results II — Agent Comparison

Algorithm	Final Reward	Avg Irrigations	Growth Deviation	Verdict
DQN	−57.41	45.0	0.14	**Optimal policy**
PPO	−73.58	47.2	0.21	Near-optimal
Q-Learning	−97.88	48.4	0.28	Tabular ceiling
SARSA	~ -105	$\sim$49	$\sim$0.30	Tabular ceiling
REINFORCE-vanilla	~ -175	$\sim$35	$\sim$0.42	High variance, unstable
REINFORCE-baseline	−338.29	0.0	0.62	Starvation
Actor-Critic	−300.96	90.0	X	Pump spammer

Results III — Algorithmic Failure Analysis (1/2)

- **DQN** wins because it is **off-policy** — its Experience Replay Buffer stores 45,000 days of past experience and trains on random samples, mathematically averaging out weather anomalies and learning a generalised policy that on-policy methods cannot match
- **PPO** — on-policy: only learns from recently collected data; short-term memory cannot smooth out the high-variance stochastic weather (random droughts vs. monsoons)
- **Q-Learning** — tabular binning destroys the smooth Gaussian gradients; the agent literally cannot see the continuous Weighted Generalised Mean curves, causing blind over- or under-watering
- **SARSA** — same binning bottleneck as Q-Learning; additionally over-conservative because on-policy exploration risks triggering the -5.0 hardware penalty

Results III — Algorithmic Failure Analysis (2/2)

- **REINFORCE-vanilla** — pure Monte Carlo: waits until episode end to update the Gt; our env needs precise daily management, we cannot tell on which day caused the plant to die on Day 70
- **REINFORCE-baseline** — discovered the starvation loophole: 0 pumps avoids all water and hardware penalties entirely; converged to total inaction as a local minimum
- **Actor-Critic**(single worker)— critic's value approximation oscillated wildly under extreme weather variance; panicked actor compensates by spamming all 90 pumps, absorbing -5.0 hardware penalties indefinitely

Analysis

What the results tell us

- You cannot bucket soil moisture values like 0.553 or 0.612 into a Q-table without losing precision, that rounding is why tabular agents plateaued and Deep RL agents did not
- The +100 harvest bonus was the only thing that made agents care about Day 89 without it, letting the plant die on Day 1 was mathematically cheaper than 90 days of water costs

What DQN actually figured out

- **Spring:** Pump moderately to build up soil moisture from the start
- **Drought:** Pump more frequently but track the tank so it learned to never let it hit empty
- **Monsoon:** When rain forecast is high, so stop pumping the sky waters the crop for free
- **Days 85–89:** Push soil to optimum and hold it there to guarantee the harvest payout

Challenge I: Reward Hacking, We Had to Fix

- **#1 — Pump Spam:** No tank limit meant the agent flooded the soil every step. We added a virtual tank that drains 5% per pump and penalises -5.0 for pumping on empty, forcing it to ration water like a real system.
- **#2 — Starvation Loop:** A small -0.05 water penalty made the agent decide never to irrigate. We fixed this with a $+100$ harvest bonus on Day 89, making survival the only rational long-term strategy.
- **#3 — Stunting the Plant:** Using `abs(actual - expected)` punished the agent for growing the plant *too well*. It learned to deliberately starve the crop to avoid deviation penalties. We split this into two signals: penalise underperformance, reward overperformance.

Challenges & Limitations II

Current limitations

- Only implemented A2C (for single thread), not tried for A3C multi-thread yet
- Action space is binary — pump ON/OFF only, no variable flow rate
- Coordinates are for North Bangalore, not for Chaparkallu from the OpenWeather api

```
failure_deviation = max(0.0, expected - actual_growth)  # Punishes dying plants
success_bonus     = max(0.0, actual_growth - expected)  # Rewards over-performing plants
```

Figure: Split into 2 signal

Conclusion

Key learnings

- Deep RL is the only viable approach for continuous state-spaces — tabular methods simply cannot compete
- Reward shaping is harder than the RL itself — agents exploit every gap you leave
- Training and production must use identical physics — any mismatch causes the agent to hallucinate

Future work

- Connect YOLO vision data directly into the state vector
- Move to A3C — multiple agents, one shared water critic
- Continuous action space for variable pump flow control

DQN — reward -57.41, 45 irrigations/episode, growth deviation 0.14 — deployed live on AWS EC2 $\rightarrow$ MQTT $\rightarrow$ ESP32.

Thank You

Questions Welcome

7-Agent RL Showdown

Custom FarmEnv Simulator

Live IoT Deployment